Revisions CSC 8604 — Cloud Computing

Fiche COMPLETE et DETAILLEE — Exam 90 min sans documents — QCM + Comprehension

1. Introduction au Cloud Computing

Definition

Le cloud computing = livraison de ressources informatiques (serveurs, stockage, bases de donnees, reseau) **a la demande via Internet** avec une **tarification a l'usage** (pay-as-you-go).

Les 6 avantages du cloud (a connaitre par coeur)

#	Avantage	Explication detaillee
1	**Depenses variables**	Au lieu d'investir dans un datacenter (CapEx), tu paies uniquement ce que tu consommes (OpEx). Pas de cout initial.
2	**Economies d'echelle**	AWS achete des centaines de milliers de serveurs → cout unitaire tres bas → repercute sur les clients
3	**Plus besoin de deviner la capacite**	On-premise : tu achetes pour le pic → gaspillage. Cloud : tu scales exactement selon la demande
4	**Vitesse et agilite**	Lancer un serveur en 2 min au lieu d'attendre 2 semaines pour commander du hardware
5	**Fin de la maintenance datacenter**	Plus besoin de gerer le refroidissement, l'alimentation, les cables, la securite physique
6	**Deploiement mondial en minutes**	Tu peux deployer ton app dans 30+ regions dans le monde en quelques clics

3 modeles de service (IaaS / PaaS / SaaS)

Modele	Toi tu geres	AWS gere	Exemples AWS
IaaS	OS, apps, donnees, middleware	Hardware, reseau, virtualisation	EC2, VPC, EBS
PaaS	Ton code et tes donnees	OS, serveur, scaling, patching	Elastic Beanstalk, RDS
SaaS	Rien (tu utilises)	Absolument tout	Gmail, Salesforce, Dropbox

Astuce QCM : Plus tu montes vers SaaS, moins tu as de controle mais moins tu as de travail.

3 modeles de deploiement

- **Cloud public** — Tout chez un fournisseur (AWS, Azure, GCP). Le plus courant, le plus flexible
- **Cloud prive (on-premises)** — Infrastructure dans ton propre datacenter avec des outils cloud (ex: VMware, OpenStack)

- **Cloud hybride** — Mix des deux. Ex: donnees sensibles en prive, reste dans le cloud public. AWS propose **AWS Outposts** pour ca

Infrastructure mondiale AWS

Concept	Definition	Nombre approx.
Region	Zone geographique isolee (ex: us-east-1 = Virginie du Nord)	30+ regions
Availability Zone (AZ)	1 ou plusieurs datacenters dans une region, relies par reseau rapide	3-6 AZ par region
Edge Location	Point de presence pour CloudFront (CDN) et Route 53	400+

Important : Chaque **region** a au minimum **2 AZ**. Les AZ sont **physiquement separees** (different batiment, alimentation, reseau) mais connectees par fibre tres rapide (latence < 1ms). C'est ca qui permet le Multi-AZ.

QCM : Quel est l'avantage principal du cloud par rapport a un datacenter sur site ?

R : Remplacer les depenses initiales (CapEx) par des depenses variables (OpEx) a l'usage

QCM : Combien d'AZ minimum par region AWS ?

R : 2 (et generalement 3 ou plus)

2. Well-Architected Framework (6 piliers)

Le WAF est le guide officiel AWS pour concevoir des architectures fiables, securisees et efficaces.

Pilier	Question cle	Principes	Services AWS
1. Excellence operationnelle	Comment executer et surveiller ?	Automatiser, petits changements, anticiper les pannes, apprendre des incidents	CloudFormation, CloudWatch, Config, Systems Manager
2. Securite	Comment proteger ?	Moindre privilege, tracabilite, securite a tous les niveaux, automatiser la securite	IAM, CloudTrail, KMS, Security Groups, WAF
3. Fiabilite	Comment recuperer des pannes ?	Tester la recuperation, scaler horizontalement, arreter de deviner la capacite	Multi-AZ, ASG, ELB, Route 53
4. Efficacite des performances	Comment utiliser les ressources efficacement ?	Utiliser les technologies avancees comme un service, deployer partout en minutes	Auto Scaling, ElastiCache, CloudFront, Lambda
5. Optimisation des couts	Comment depenser le minimum ?	Adopter un modele de consommation, mesurer l'efficacite, arreter les depenses inutiles	Cost Explorer, Budgets, Reserved Instances, Spot
6. Durabilite	Comment minimiser	Comprendre son impact, maximiser	Rightsizing, Graviton,

l'impact environnemental ?	l'utilisation, adopter du managed	Lambda

QCM : Quel pilier du WAF concerne la capacite a recuperer automatiquement d'une panne ?

R : Pilier Fiabilite (Reliability)

3. VPC — Virtual Private Cloud (RESEAU) — EN DETAIL

C'est quoi un VPC ?

Un VPC est **ton propre reseau prive virtuel dans AWS**. Tu as le controle total sur :

- La plage d'adresses IP (CIDR block)
- La creation de subnets
- La configuration des tables de routage
- Les passerelles reseau (IGW, NAT)
- Les regles de securite (SG, NACL)

> ***VPC** = ton immeuble prive | **Subnets** = les etages | **IGW** = la porte d'entree vers la rue | **NAT** = fenetre pour voir dehors sans etre vu | **Route Table** = panneaux directionnels | **SG** = vigile a chaque porte | **NACL** = vigile a l'entree de chaque etage*

CIDR Block (les adresses IP)

Quand tu crees un VPC, tu choisis une plage d'adresses IP en notation **CIDR**.

CIDR	Nombre d'IPs	Utilisation
`10.0.0.0/16`	65 536 IPs	VPC principal (le plus courant)
`10.0.1.0/24`	256 IPs	Subnet (256 - 5 reservees = 251 utilisables)
`10.0.2.0/24`	256 IPs	Autre subnet

Regle : Le /16 c'est le VPC (large), les /24 c'est les subnets (plus petits). Plus le nombre apres / est grand, moins il y a d'IPs.

AWS reserve 5 IPs par subnet : la premiere (reseau), la derniere (broadcast), et 3 pour AWS (DNS, routeur, futur usage).

Subnets en detail

Un subnet = un sous-reseau dans UNE seule AZ. Un subnet ne peut PAS s'etendre sur plusieurs AZ.

Mais tu peux avoir plusieurs subnets dans la meme AZ.

	Subnet PUBLIC	Subnet PRIVE
Route table	Route `0.0.0.0/0 → igw-xxx`	Route `0.0.0.0/0 → nat-xxx` (ou

		aucune)
IP publique ?	Oui (auto-assign public IP active)	Non (IP privee uniquement)
Accessible depuis Internet ?	Oui (si SG le permet)	Non
Peut sortir vers Internet ?	Oui (via IGW)	Oui seulement via NAT Gateway
On y met quoi ?	ALB, Bastion Host, NAT Gateway	EC2 appli, RDS, ElastiCache

Internet Gateway (IGW)

- C'est la **porte d'entree/sortie vers Internet** pour ton VPC
- Il y a **1 IGW par VPC** (pas plus)
- Il est **hautement disponible** par defaut (AWS le gere)
- Pour qu'un subnet soit public, sa route table doit avoir : `0.0.0.0/0 → igw-xxx`

NAT Gateway

- Permet aux instances dans un **subnet prive** d'acceder a Internet (ex: pour telecharger des mises a jour)
- **SANS que Internet puisse les contacter** (sens unique : sortie seulement)
- Se place dans un **subnet public** (il a besoin de l'IGW)
- Coute de l'argent (~$0.045/h + cout du trafic)
- Pour la haute dispo : un NAT Gateway par AZ

Schema logique du flux reseau

```
Instance privee → Route Table (0.0.0.0/0 → NAT) → NAT Gateway (dans subnet public) →
```

```
Route Table (0.0.0.0/0 → IGW) → Internet Gateway → Internet
```

Route Tables (tables de routage)

- Chaque subnet est associe a **exactement 1 route table**
- Une route table contient des **regles de routage** (destination → cible)
- Route table publique : `10.0.0.0/16 → local` + `0.0.0.0/0 → igw-xxx`
- Route table privee : `10.0.0.0/16 → local` + `0.0.0.0/0 → nat-xxx`
- La route `local` permet la communication entre tous les subnets du VPC

Elastic IP (EIP)

- Adresse IP publique **statique** (ne change pas au reboot)
- Tu peux l'attacher a une EC2 ou un NAT Gateway
- **Gratuite quand elle est utilisee**, payante si elle ne l'est pas (AWS ne veut pas de gaspillage d'IPs)

Security Groups (SG) — EN DETAIL

Security Group = firewall VIRTUEL au niveau de l'instance. Chaque instance EC2 a au moins 1 SG.

Propriete	Detail
Niveau	Instance (ENI = Elastic Network Interface)
Stateful	Si tu autorises le trafic ENTRANT, la reponse sort automatiquement (pas besoin de regle sortante)
Regles	Uniquement des regles **ALLOW** (pas de DENY possible)
Par defaut	Tout le trafic ENTRANT est **bloque**. Tout le trafic SORTANT est **autorise**
Evaluation	Toutes les regles sont evaluees (pas d'ordre de priorite)
Reference croisee	Tu peux autoriser un autre SG comme source (ex: SG du web → SG de la BDD)

Exemples de regles SG typiques

SG pour	Type	Protocol	Port	Source
Serveur Web	Inbound	TCP	80 (HTTP)	0.0.0.0/0 (tout le monde)
Serveur Web	Inbound	TCP	443 (HTTPS)	0.0.0.0/0
Serveur Web	Inbound	TCP	22 (SSH)	Mon IP uniquement
Base de donnees	Inbound	TCP	3306 (MySQL)	**sg-web** (le SG du serveur web)

Bonne pratique : La BDD n'accepte les connexions QUE depuis le SG du serveur web (pas depuis 0.0.0.0/0 !). C'est le **chainage de SG**.

NACL (Network ACL) — EN DETAIL

Propriete	Detail
Niveau	Subnet (s'applique a tout le trafic entrant/sortant du subnet)
Stateless	Tu dois creer des regles pour le trafic ENTRANT **ET** SORTANT separement
Regles	ALLOW **et DENY** possibles
Par defaut	La NACL par defaut **autorise tout**. Les NACL custom **bloquent tout**
Evaluation	Les regles sont evaluees **dans l'ordre des numeros** (de la plus petite a la plus grande). La premiere qui matche gagne
Numerotation	Regles numerotees (ex: 100, 200...). Convention : incrementer par 100 pour pouvoir inserer

SG vs NACL — Tableau comparatif COMPLET

Critere	Security Group	NACL
Niveau	Instance (ENI)	Subnet
Stateful / Stateless	**Stateful**	**Stateless**
Regles	ALLOW uniquement	ALLOW + DENY
Evaluation	Toutes les regles	Dans l'ordre des numeros

Par defaut (entrant)	Tout bloque	Tout autorise (defaut) / tout bloque (custom)
Par defaut (sortant)	Tout autorise	Tout autorise (defaut)
S'applique a	Seulement si associe a l'instance	Automatiquement a tout le subnet

VPC Peering

- Connexion reseau **directe entre 2 VPC** (meme compte ou differents, meme region ou differentes)
- **Non transitif** : si VPC A ↔ VPC B et VPC B ↔ VPC C, alors A ne peut PAS parler a C (il faut un peering A ↔ C)
- Les CIDR des 2 VPC ne doivent **pas se chevaucher**

QCM : Un serveur dans un subnet prive doit telecharger des mises a jour depuis Internet. Quel composant est necessaire ?

R : NAT Gateway (place dans un subnet public)

QCM : Tu veux que ta BDD MySQL n'accepte des connexions que depuis ton serveur web EC2. Comment configurer le SG de la BDD ?

R : Inbound rule : TCP port 3306, source = le Security Group du serveur web (sg-xxx)

QCM : Quelle est la difference fondamentale entre Stateful et Stateless ?

R : Stateful (SG) = la reponse est automatiquement autorisee. Stateless (NACL) = tu dois autoriser entree ET sortie explicitement

QCM : Un VPC A est peere avec B, et B est peere avec C. A peut-il communiquer avec C ?

R : Non. Le VPC peering est non transitif. Il faut un peering direct A ↔ C

4. EC2 — Elastic Compute Cloud — EN DETAIL

C'est quoi ?

EC2 = **machine virtuelle (serveur) que tu loues a la demande**. Tu choisis l'OS, la taille, le stockage, et le reseau.

Les composants cles d'une instance EC2

Composant	Explication detaillee

AMI (Amazon Machine Image)	Image/photo d'un serveur (OS + logiciels pre-installes). Tu peux utiliser une AMI AWS, du Marketplace, ou **creer la tienne** a partir d'une instance existante. Utile pour dupliquer un serveur configure
Instance Type	La "taille" du serveur : nombre de vCPU, RAM, stockage, reseau. Format : `famille.taille` (ex: t3.micro)
Key Pair	Paire de cles SSH (publique/privee) pour se connecter a l'instance Linux. La cle privee = ton mot de passe
Security Group	Firewall virtuel (voir section VPC). Tu peux en attacher plusieurs a une meme instance
User Data	Script bash qui s'execute **une seule fois au premier demarrage**. Sert a installer des paquets, configurer le serveur, telecharger du code
IAM Role	Permissions AWS attachees a l'instance. Ex: droit de lire un secret dans Secrets Manager. **TOUJOURS utiliser un role, JAMAIS mettre des cles dans le code**
EBS Volume	Le disque dur de l'instance (voir section stockage)
Elastic IP	IP publique fixe (ne change pas quand tu redemarres l'instance)

Familles d'instances (a connaitre)

Famille	Lettre	Optimise pour	Cas d'usage
General Purpose	T, M	Equilibre CPU/RAM	Sites web, apps, petites BDD
Compute Optimized	C	CPU puissant	Calcul scientifique, encoding video, gaming
Memory Optimized	R, X	Beaucoup de RAM	Grosses BDD en memoire, caches
Storage Optimized	I, D, H	I/O disque rapide	Data warehousing, fichiers distribues
Accelerated	P, G, Inf	GPU / TPU	Machine learning, rendu graphique

T3.micro fait partie du **Free Tier** (750h/mois pendant 12 mois). C'est une instance "burstable" (elle accumule des credits CPU quand elle est idle).

Modes d'achat — EN DETAIL

Mode	Engagement	Reduction	Interruption ?	Quand l'utiliser
On-Demand	Aucun	0%	Non	Dev/test, charge imprevisible, court terme
Reserved (RI)	1 an ou 3 ans	Jusqu'a -72%	Non	Production stable 24/7 (serveur de BDD, appli critique)
Savings Plans	1 an ou 3 ans	Jusqu'a -72%	Non	Comme RI mais plus flexible (pas lie a un type d'instance)
Spot Instances	Aucun	Jusqu'a -	**OUI** (2 min de	Batch processing, big data, CI/CD, calcul non

		90%	preavis)	critique
Dedicated Hosts	Variable	Variable	Non	Compliance, licences logicielles liees au hardware (Oracle, SQL Server)
Dedicated Instances	Variable	Variable	Non	Isolation physique mais pas de controle du placement

User Data — Script de demarrage

Le User Data s'execute en tant que **root** au **premier lancement uniquement**. Exemple typique :

```
#!/bin/bash
```

```
yum update -y
```

```
yum install -y httpd
```

```
systemctl start httpd
```

```
systemctl enable httpd
```

```
echo "Hello World" > /var/www/html/index.html
```

Metadata de l'instance

Chaque instance EC2 peut acceder a ses propres infos via : `http://169.254.169.254/latest/meta-data/`

Exemples : IP publique, IP privee, IAM role, instance-id, AZ, etc.

Placement Groups

Type	Comportement	Cas d'usage
Cluster	Toutes les instances dans le meme rack	HPC, faible latence
Spread	Chaque instance sur un hardware different	Haute dispo critique (max 7 par AZ)
Partition	Groupes sur des racks differents	Big data (Hadoop, Kafka)

QCM : Un client veut reduire ses couts EC2 de 60% pour une charge previsible qui tourne 24/7. Que recommander ?

R : Reserved Instances ou Savings Plans (engagement 1-3 ans = grosse reduction)

QCM : Un script doit s'executer automatiquement au premier demarrage d'une EC2. Quelle fonctionnalite utiliser ?

R : EC2 User Data

QCM : Quel type d'instance pour une application de machine learning qui a besoin de GPU ?

R : Accelerated Computing (famille P ou G)

QCM : Comment donner a une EC2 la permission de lire un secret dans Secrets Manager ?

R : Attacher un IAM Role a l'instance avec la policy permettant secretsmanager:GetSecretValue

5. Stockage — S3, EBS, EFS — EN DETAIL

Amazon S3 (Simple Storage Service)

S3 = **stockage objet illimite**. Tu mets des fichiers (objets) dans des buckets.

Concepts cles S3

Concept	Detail
Bucket	Conteneur de niveau superieur. Nom **unique au monde**. Cree dans une region specifique
Objet	Un fichier + metadonnees. Taille max : **5 To** par objet
Cle (Key)	Le chemin complet de l'objet dans le bucket (ex: images/photo.jpg)
Durabilite	**99.999999999%** (11 neufs). Tes fichiers ne seront JAMAIS perdus
Disponibilite	99.99% pour S3 Standard (= ~53 min d'indisponibilite/an max)
Versioning	Garder toutes les versions d'un objet. Protection contre la suppression accidentelle
Encryption	Chiffrement au repos par defaut (SSE-S3). Options : SSE-KMS, SSE-C

Classes de stockage S3 (du plus cher au moins cher)

Classe	Acces	Cout stockage	Cout recuperation	Cas d'usage
S3 Standard	Frequent	$$$	Gratuit	Site web, images, donnees actives
S3 Intelligent-Tiering	Variable	$$	Gratuit	Acces imprevisible (AWS deplace auto)
S3 Standard-IA	Peu frequent	$$	Payant	Backups, donnees qu'on garde "au cas ou"
S3 One Zone-IA	Peu frequent	$	Payant	Donnees re-creables (1 seule AZ = moins resilient)
S3 Glacier Instant	Rare	$	Payant	Archive avec acces en millisecondes
S3 Glacier Flexible	Tres rare	¢	Minutes a heures	Archives longue duree
S3 Glacier Deep Archive	Quasi jamais	¢¢	**12 heures**	Compliance, archivage legal 7+ ans

S3 Lifecycle Rules

Tu peux automatiquement deplacer les objets entre classes au fil du temps :

```
Standard → (apres 30j) → Standard-IA → (apres 90j) → Glacier → (apres 365j) → Deep Archive
```

S3 Static Website Hosting

- S3 peut heberger un **site web statique** (HTML, CSS, JS) sans serveur
- URL format : `http://bucket-name.s3-website-region.amazonaws.com`
- Il faut activer le static website hosting ET rendre les objets publics (bucket policy)

S3 Bucket Policy vs ACL

- **Bucket Policy** = JSON qui definit qui peut acceder au bucket et comment. S'applique au bucket entier
- **ACL** = permissions plus simples et plus anciennes. AWS recommande de les desactiver
- **Block Public Access** = parametres au niveau du compte/bucket pour empecher tout acces public accidentel

Amazon EBS (Elastic Block Store)

EBS = **disque dur virtuel** attache a une instance EC2. C'est le stockage principal de l'instance.

Propriete	Detail
Attache a	**1 seule instance EC2** dans la **meme AZ**
Persistence	Les donnees persistent meme si l'instance est arretee (mais pas terminee par defaut pour le root)
Snapshots	Backup du volume EBS, stocke dans S3. Peut etre copie dans une autre region
Types	gp3 (usage general SSD), io2 (hautes performances), st1 (HDD throughput), sc1 (cold HDD)
Chiffrement	AES-256, gere par AWS KMS. Transparent pour l'utilisateur

Types de volumes EBS

Type	Techno	Cas d'usage	Performance
gp3 / gp2	SSD	Boot volume, dev, apps generales	Jusqu'a 16 000 IOPS
io2 / io1	SSD	BDD haute performance (Oracle, SQL Server)	Jusqu'a 64 000 IOPS
st1	HDD	Big data, logs, streaming	Throughput optimise
sc1	HDD	Donnees froides, archivage	Le moins cher

Amazon EFS (Elastic File System)

- Systeme de fichiers **partage** accessible par **plusieurs instances EC2 en meme temps**
- Protocole **NFS** (Linux uniquement)
- **Scalabilite automatique** (grandit/diminue selon tes fichiers)
- Fonctionne **a travers les AZ** d'une region
- Cas d'usage : partage de fichiers, CMS, home directories

Comparaison S3 vs EBS vs EFS

	S3	EBS	EFS
Type	Stockage objet	Block storage (disque dur)	File system (NFS)
Acces	Via HTTP/API	Monte sur 1 EC2	Monte sur N EC2
AZ	Multi-AZ auto	1 seule AZ	Multi-AZ
Taille max	Illimitee	16 To par volume	Illimitee
Cas	Fichiers, backups, site statique	OS, BDD, boot volume	Fichiers partages entre serveurs

QCM : Quel stockage pour partager des fichiers entre 10 instances EC2 reparties dans 3 AZ ?

R : Amazon EFS (seul capable d'etre monte sur plusieurs instances dans plusieurs AZ)

QCM : Comment reduire le cout de stockage pour des fichiers consultes rarement ?

R : Utiliser S3 Lifecycle Rules pour deplacer vers S3 Glacier ou Deep Archive

6. RDS — Bases de donnees — EN DETAIL

Amazon RDS (Relational Database Service)

RDS = **base de donnees relationnelle geree par AWS**. AWS gere : patching, backups, haute dispo, monitoring.

Moteurs supportes

MySQL, PostgreSQL, MariaDB, Oracle, SQL Server, **Amazon Aurora**

Avantages de RDS vs BDD sur EC2

Fonctionnalite	MySQL sur EC2	Amazon RDS
Installation	Manuelle	Quelques clics
Patching OS	Toi	AWS
Backups	A configurer	Automatiques (retention 1-35 jours)
Haute dispo	A configurer	Multi-AZ en 1 clic
Scaling	Migration manuelle	Vertical scaling + Read Replicas
Monitoring	A installer	CloudWatch integre
Acces SSH	Oui	**Non** (service manage)

Multi-AZ vs Read Replicas (QUESTION EXAM FREQUENTE)

	Multi-AZ	Read Replica
Objectif	**HAUTE DISPONIBILITE**	**PERFORMANCE EN LECTURE**
Comment	Copie synchrone dans une autre AZ	Copie asynchrone (peut etre dans une autre region)
Failover	Automatique (DNS bascule)	Pas de failover automatique
Lecture directe ?	**NON** (standby invisible)	**OUI** (endpoint de lecture dedie)
Nombre max	1 standby	Jusqu'a 5 replicas
Cout	Double (2 instances)	Cout par replica supplementaire

Amazon Aurora

- Version AWS de MySQL/PostgreSQL, **5x plus rapide que MySQL, 3x que PostgreSQL**
- Stockage auto-scale de 10 Go a 128 To
- **6 copies des donnees sur 3 AZ** automatiquement
- Jusqu'a 15 Read Replicas (vs 5 pour RDS classique)
- **Aurora Serverless** : scaling automatique, paie a l'usage

Amazon DynamoDB

- BDD **NoSQL** (cle-valeur et document) geree par AWS
- **Serverless** (pas de serveur a gerer), scaling automatique
- Latence en **millisecondes** a n'importe quelle echelle
- Modele : Table → Items (lignes) → Attributes (colonnes)
- Cle primaire obligatoire : **Partition Key** (+ optionnel : Sort Key)
- Capacite : On-Demand (auto) ou Provisioned (tu fixes les Read/Write Units)
- **DAX** = cache en memoire pour DynamoDB (latence microseconde)

QCM : Comment rendre une BDD RDS hautement disponible en cas de panne d'une AZ ?

R : Activer Multi-AZ deployment (failover automatique vers le standby)

QCM : L'application fait beaucoup de lectures. Comment reduire la charge sur la BDD principale ?

R : Creer des Read Replicas et diriger les lectures vers elles

QCM : Quel service de BDD pour des donnees cle-valeur avec une latence < 10ms et un scaling automatique ?

R : Amazon DynamoDB

7. IAM — Identity and Access Management — EN DETAIL

Les 4 composants

Composant	Definition	Detail
User	Une personne ou application	A des credentials (password pour console, access key pour CLI/API). **1 user = 1 personne**
Group	Collection d'utilisateurs	Les policies attachees au groupe s'appliquent a tous ses membres. Ex: groupe "Developers"
Role	Identite temporaire	Pour les **services AWS** (EC2, Lambda) ou les utilisateurs federes. Pas de credentials permanentes
Policy	Document JSON de permissions	Definit Effect (Allow/Deny) + Action (s3:GetObject) + Resource (arn:aws:s3:::mon-bucket/*)

Structure d'une policy IAM

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": "s3:GetObject",
    "Resource": "arn:aws:s3:::mon-bucket/*"
  }]
}
```

Cette policy autorise la lecture de tous les objets dans "mon-bucket".

Types de policies

Type	Gere par	Exemple
AWS Managed	AWS (precreee)	AmazonS3ReadOnlyAccess
Customer Managed	Toi	Policy custom pour ton application
Inline	Toi, attachee a 1 seul user/group/role	Permission tres specifique pour 1 ressource

Principes de securite IAM

1. **Principe du moindre privilege** — Donner uniquement les permissions strictement necessaires

2. **Ne JAMAIS utiliser le compte root** pour les taches quotidiennes — Creer un IAM admin a la place
3. **Activer MFA** (Multi-Factor Authentication) sur root et les comptes sensibles
4. **Utiliser des Roles pour les services** — EC2 qui accede a S3 → IAM Role, pas des access keys dans le code
5. **Rotation des cles** — Changer regulierement les access keys
6. **Audit avec CloudTrail** — Chaque appel API est logue

Role vs User — Quand utiliser quoi ?

Situation	Utiliser	Pourquoi
Personne qui se connecte a la console	IAM User	Credentials permanentes (password + MFA)
EC2 qui doit acceder a S3	IAM Role	Credentials temporaires, pas de cles dans le code
Lambda qui ecrit dans DynamoDB	IAM Role	Service AWS → toujours un role
Application externe qui accede a AWS	IAM Role + STS	Federation, credentials temporaires

Modele de responsabilite partagee

AWS est responsable de la securite DU cloud :

- Hardware physique, datacenters, reseau global
- Virtualisation, hyperviseur
- Regions, AZ, Edge Locations

TOI tu es responsable de la securite DANS le cloud :

- Configuration des SG, NACL
- Gestion IAM (users, roles, policies)

•Chiffrement des donnees

•Patching de l'OS (si EC2 = IaaS)

•Configuration du firewall (SG, NACL)

•Formation des utilisateurs

QCM : Une instance EC2 a besoin de lire des objets dans un bucket S3. Quelle est la methode la plus securisee ?

R : Creer un IAM Role avec la policy s3:GetObject et l'attacher a l'instance EC2

QCM : Un developpeur a accidentellement rendu public un bucket S3. Qui est responsable ?

R : Le client (toi). La configuration des ressources est ta responsabilite (modele de responsabilite partagee)

QCM : Quel outil AWS permet de tracer chaque appel API effectue sur le compte ?

R : AWS CloudTrail

8. Load Balancing + Auto Scaling — EN DETAIL

Elastic Load Balancer (ELB)

Un load balancer **repartit le trafic entre plusieurs instances** pour eviter la surcharge d'un seul serveur.

Types de Load Balancer

Type	Couche OSI	Protocoles	Fonctionnalites	Cas d'usage
ALB	Couche 7	HTTP, HTTPS, WebSocket	Routage par URL, host, header. Target groups	Sites web, API REST, microservices
NLB	Couche 4	TCP, UDP, TLS	Ultra-rapide (millions req/s), IP statique	Gaming, IoT, performance extreme
GLB	Couche 3	IP	Routage vers appliances virtuelles	Firewalls, IDS/IPS

Concepts ALB

- **Listener** : regle qui ecoute sur un port (ex: port 80 → redirect vers HTTPS, port 443 → forward vers target

group)

- **Target Group** : groupe d'instances qui recevront le trafic. Avec **health checks**
- **Health Check** : l'ALB ping regulierement chaque instance (ex: GET /health). Si elle ne repond pas, l'ALB arrete de lui envoyer du trafic
- **Cross-zone load balancing** : repartit le trafic uniformement entre toutes les AZ
- **Sticky Sessions** : un utilisateur est toujours envoye sur la meme instance (cookie)

Auto Scaling Group (ASG)

L'ASG **ajoute ou retire automatiquement des instances EC2** selon la demande.

Composants d'un ASG

Composant	Role
Launch Template	Modele d'instance (AMI, type, SG, user data, IAM role). Remplace le Launch Configuration
Capacite min	Nombre minimum d'instances (jamais en dessous)
Capacite max	Nombre maximum d'instances (ne depassera jamais)
Desired capacity	Nombre d'instances souhaite a l'instant T
Scaling Policy	Regle qui declenche le scaling

Types de Scaling Policies

Type	Comment ca marche	Exemple
Target Tracking	Maintenir une metrique a une valeur cible	"Garder le CPU moyen a 50%"
Step Scaling	Ajouter/retirer X instances selon des seuils	"CPU > 70% → +2 instances, CPU > 90% → +4"
Simple Scaling	Ajouter/retirer X instances sur un seuil unique	"CPU > 80% → +1 instance"
Scheduled	Planifie a l'avance	"Tous les lundis a 8h → 10 instances"
Predictive	ML analyse les patterns passes	Anticipe les pics de trafic

ALB + ASG ensemble

```
Utilisateurs → ALB → Target Group → ASG (min=2, max=10, desired=2)
```

Quand le trafic augmente : ASG lance de nouvelles instances → les enregistre dans le target group → ALB repartit le trafic dessus. Quand le trafic baisse : ASG termine les instances superflues.

Scalabilite verticale vs horizontale

	Verticale (Scale Up)	Horizontale (Scale Out)
Quoi	Plus gros serveur (plus de CPU/RAM)	Plus de serveurs

Limite	Taille max du hardware	Pas de limite
Downtime	Oui (il faut arreter pour changer de taille)	Non (on ajoute)
AWS	Changer instance type (t3.micro → t3.large)	Auto Scaling Group

En QCM, la bonne reponse est presque toujours le scaling HORIZONTAL (ASG).

QCM : Comment gerer automatiquement les pics de trafic pour un site web ?

R : ALB + Auto Scaling Group avec une target tracking policy (ex: CPU a 50%)

QCM : Le health check de l'ALB echoue pour une instance. Que se passe-t-il ?

R : L'ALB retire l'instance du target group (plus de trafic envoye). L'ASG peut la terminer et en lancer une nouvelle

QCM : Quelle est la difference entre un ALB et un NLB ?

R : ALB = couche 7 (HTTP/HTTPS, routage par URL). NLB = couche 4 (TCP/UDP, ultra-performant)

9. CloudFormation — Infrastructure as Code

Principe

CloudFormation = **decrire TOUTE ton infrastructure dans un fichier texte** (YAML ou JSON). AWS cree/modifie/supprime les ressources automatiquement.

Concepts cles

Concept	Definition
Template	Le fichier YAML/JSON qui decrit les ressources a creer
Stack	L'ensemble des ressources creees a partir d'un template. 1 template → 1 stack
Change Set	Preview des modifications avant de les appliquer
Drift Detection	Detecter si quelqu'un a modifie manuellement une ressource creee par CloudFormation
Rollback	Si une creation echoue → tout est automatiquement supprime (retour a zero)

Sections d'un template CloudFormation

Section	Obligatoire ?	Role
AWSTemplateFormatVersion	Non	Version du format (toujours "2010-09-09")
Description	Non	Description du template

Parameters	Non	Valeurs dynamiques (ex: type d'instance, nom du bucket)
Resources	**OUI**	Les ressources AWS a creer (EC2, VPC, RDS...)
Outputs	Non	Valeurs retournees apres creation (ex: URL du load balancer)

Avantages IaC

- **Reproductible** : deployer la meme infra dans dev, staging, prod
- **Versionnable** : le template est dans Git, on peut voir l'historique
- **Automatise** : pas d'erreur humaine de clic dans la console
- **Documentation vivante** : le template IS la documentation de l'infra
- **Rollback** : si ca echoue, tout est annule

QCM : Quel service deploie une infrastructure de facon reproductible et versionnee ?

R : AWS CloudFormation (Infrastructure as Code)

QCM : Quelle section est obligatoire dans un template CloudFormation ?

R : Resources

10. Mise en cache — CloudFront et ElastiCache

Amazon CloudFront (CDN)

CloudFront = **Content Delivery Network**. Des copies de ton contenu dans 400+ points de presence dans le monde.

Comment ca marche

1. L'utilisateur demande une page web
2. CloudFront cherche dans l'edge location la plus proche
3. Si le contenu est en cache → le retourne immediatement (**cache hit**)
4. Si non en cache → va le chercher a l'**origin** (S3, EC2, ALB) → le met en cache → le retourne

Concepts cles

Concept	Definition
Distribution	La configuration CloudFront (origin, comportement du cache, TTL)
Origin	La source du contenu (S3 bucket, EC2, ALB, HTTP externe)
Edge Location	Point de presence ou le contenu est cache
TTL	Time to Live : combien de temps le contenu reste en cache
Invalidation	Forcer la suppression du cache (payant)

OAC/OAI	Permettre a CloudFront d'acceder a un bucket S3 prive

Cas d'usage typique : S3 (site statique) + CloudFront = site web rapide dans le monde entier avec HTTPS gratuit (certificat ACM).

Amazon ElastiCache

ElastiCache = **cache en memoire RAM** pour les requetes de BDD frequentes.

	Redis	Memcached
Persistence	Oui (donnees sauvees sur disque)	Non (tout en memoire)
Replication	Oui (Multi-AZ, Read Replicas)	Non
Types de donnees	Strings, listes, sets, hashes	Strings uniquement
Cas d'usage	Sessions, classements, queues	Cache simple

QCM : Comment reduire la latence pour des utilisateurs en Asie accedant a un site heberge en Europe ?

R : Amazon CloudFront (cache le contenu dans les edge locations proches de l'Asie)

QCM : La BDD est surchargee par des requetes repetitives. Comment reduire la charge ?

R : Ajouter Amazon ElastiCache (Redis ou Memcached) devant la BDD

11. Architectures decouplees — SQS, SNS, Lambda

Amazon SQS (Simple Queue Service)

- **File d'attente de messages** geree par AWS
- Producteur envoie un message → SQS le stocke → Consommateur le lit et le supprime
- **Standard Queue** : debit illimite, au moins une livraison, ordre non garanti
- **FIFO Queue** : ordre garanti, exactement une livraison, 300 msg/s max
- **Visibility Timeout** : temps pendant lequel un message lu est invisible aux autres consommateurs
- **Dead Letter Queue (DLQ)** : queue ou vont les messages qui echouent apres N tentatives
- Retention : 1 minute a **14 jours** (defaut : 4 jours)

Amazon SNS (Simple Notification Service)

- **Pub/Sub** : 1 message → N abonnes
- Un **topic** recoit des messages et les distribue a tous les abonnes
- Abonnes possibles : SQS, Lambda, email, SMS, HTTP endpoint
- Cas d'usage : notifications, alertes, fan-out (1 event → plusieurs actions)

Pattern SNS + SQS (fan-out)

```
Evenement → SNS Topic → SQS Queue 1 (traitement commande)
                      → SQS Queue 2 (envoi email)
                      → SQS Queue 3 (analytics)
```

Un seul message publie sur SNS est distribue a 3 queues SQS independantes.

AWS Lambda

- Executer du code **sans serveur** (serverless). Tu upload ton code, AWS l'execute
- **Pay per invocation** : tu paies uniquement quand le code tourne (nombre de requetes + duree)
- Langages : Python, Node.js, Java, Go, .NET, Ruby
- Duree max : **15 minutes** par execution
- Memoire : 128 Mo a 10 Go
- Triggers : API Gateway, S3, SQS, SNS, CloudWatch Events, DynamoDB Streams...

QCM : Comment decoupler un service de traitement de commandes d'un service d'envoi d'email ?

R : Mettre une queue SQS entre les deux. Le service commande envoie un message, le service email le consomme

QCM : Quel service pour executer du code en reponse a un upload S3, sans gerer de serveur ?

R : AWS Lambda (triggered par un evenement S3)

12. Monitoring et Gestion — CloudWatch, CloudTrail, Config

Amazon CloudWatch

Fonctionnalite	Detail
Metrics	Metriques de performance (CPU, memoire, disque, reseau). EC2 envoie des metriques toutes les 5 min (gratuit) ou 1 min (detailed monitoring, payant)
Alarms	Declencher une action quand une metrique depasse un seuil. Ex: CPU > 80% → envoyer email SNS ou scaling ASG
Logs	Stocker et analyser les logs de tes applications et services AWS
Dashboards	Tableaux de bord visuels personnalisables
Events / EventBridge	Reagir a des changements d'etat (ex: une instance EC2 s'arrete → lancer une Lambda)

AWS CloudTrail

- **Audit trail** : enregistre CHAQUE appel API sur ton compte AWS

- Qui a fait quoi, quand, depuis ou
- Active par defaut (90 jours de retention gratuite)
- Cas d'usage : audit de securite, compliance, investigation d'incidents

AWS Config

- Surveille et enregistre la **configuration** de tes ressources AWS au fil du temps
- Permet de verifier la **conformite** (ex: "tous les buckets S3 doivent etre chiffres")
- Garde un historique : "cette ressource a change de config le 5 mars"

AWS Systems Manager

- Gerer et configurer tes instances EC2 **a distance**
- **Session Manager** : se connecter a une instance SANS SSH (pas besoin de port 22 ouvert !)
- **Parameter Store** : stocker des parametres de configuration et secrets
- **Patch Manager** : automatiser les mises a jour de l'OS
- **Run Command** : executer des commandes sur plusieurs instances a la fois

QCM : Quel service pour surveiller le CPU d'une instance EC2 et envoyer une alerte si > 80% ?

R : Amazon CloudWatch (creer une CloudWatch Alarm)

QCM : Un utilisateur a supprime un bucket S3. Comment savoir qui l'a fait ?

R : AWS CloudTrail (enregistre tous les appels API)

QCM : Comment se connecter a une EC2 sans ouvrir le port SSH (22) ?

R : AWS Systems Manager Session Manager

13. Securite avancee

AWS Secrets Manager

- Stocker des **secrets** (mots de passe BDD, cles API) de maniere securisee
- **Rotation automatique** des secrets (ex: changer le mot de passe RDS tous les 30 jours)
- Acces via API : `GetSecretValue` (l'app demande le secret au runtime)
- C'est ce qu'on utilise dans le mini-projet pour le mot de passe de la BDD RDS

AWS KMS (Key Management Service)

- Gerer les **cles de chiffrement**
- Utilise par S3, EBS, RDS, etc. pour le chiffrement au repos
- **CMK** (Customer Master Key) : ta propre cle ou une cle geree par AWS

AWS WAF (Web Application Firewall)

- Protege les applications web contre les attaques (SQL injection, XSS, bots)

- Se place devant **CloudFront, ALB, ou API Gateway**
- Regles basees sur IP, geolocalisation, taille de requete, patterns

AWS Shield

- **Shield Standard** (gratuit) : protection DDoS basique, automatique sur tous les services
- **Shield Advanced** (payant, $3000/mois) : protection DDoS avancee, equipe de reponse, remboursement des couts lies a l'attaque

Chiffrement

Type	Quand	Exemples
Au repos (at rest)	Donnees stockees sur disque	S3 (SSE-S3, SSE-KMS), EBS, RDS
En transit	Donnees qui circulent sur le reseau	HTTPS/TLS, SSL pour RDS

AWS GuardDuty

- **Detection de menaces** intelligente
- Analyse CloudTrail, VPC Flow Logs, DNS Logs avec du machine learning
- Detecte : crypto mining, acces depuis des IPs malveillantes, comportements anormaux

AWS Inspector

- Analyse automatique de **vulnerabilites** dans tes EC2 et conteneurs
- Verifie les CVE, la configuration reseau, les bonnes pratiques

QCM : Ou stocker le mot de passe de la base de donnees pour une application Node.js sur EC2 ?

R : AWS Secrets Manager (avec un IAM Role sur l'EC2 pour y acceder)

QCM : Quel service protege une application web contre les attaques SQL injection ?

R : AWS WAF (place devant l'ALB ou CloudFront)

14. Couts et facturation — EN DETAIL

Modele de tarification AWS

- **Pay-as-you-go** : payer ce qu'on consomme (pas d'engagement par defaut)
- **Payer moins en reservant** : RI, Savings Plans
- **Payer moins a grande echelle** : reductions de volume (S3 par exemple)
- **Trafic entrant** : gratuit (data IN)
- **Trafic sortant** : payant (data OUT vers Internet)
- **Trafic entre AZ** : payant (petite charge)
- **Trafic dans la meme AZ** : gratuit (si IP privee)

Outils de gestion des couts

Outil	Role	Detail
AWS Free Tier	Services gratuits	t2.micro 750h/mois, S3 5 Go, RDS 750h/mois (pendant 12 mois)
AWS Pricing Calculator	Estimer AVANT	Simuler le cout mensuel d'une architecture
AWS Cost Explorer	Voir APRES	Visualiser et analyser les depenses passees, previsions
AWS Budgets	Alertes	Definir un budget et recevoir des alertes quand on approche/depasse
AWS Cost & Usage Report	Detail complet	Le rapport le plus detaille (par heure, par ressource)
AWS Trusted Advisor	Recommandations	5 categories : cout, performance, securite, tolerance aux pannes, limites de service

Strategies de reduction des couts

1. **Reserved Instances / Savings Plans** pour les charges stables (jusqu'a -72%)
2. **Spot Instances** pour les charges interruptibles (jusqu'a -90%)
3. **Rightsizing** : ne pas surdimensionner (utiliser CloudWatch pour analyser l'usage reel)
4. **S3 Lifecycle** : deplacer les anciens fichiers vers Glacier
5. **Eteindre les ressources non utilisees** (dev/test la nuit et le week-end)
6. **Utiliser les services manages** (RDS vs MySQL sur EC2 : moins de temps d'admin)

AWS Organizations

- Gerer **plusieurs comptes AWS** de maniere centralisee
- **Consolidated Billing** : une seule facture pour tous les comptes (= reductions de volume)
- **Service Control Policies (SCP)** : restreindre les services/actions autorisees par compte

QCM : Quel outil pour estimer le cout d'une architecture AVANT de la deployer ?

R : AWS Pricing Calculator

QCM : Le trafic entrant dans AWS est-il payant ?

R : Non, le trafic entrant (data IN) est gratuit. Seul le trafic sortant (data OUT) est payant

15. Autres services importants

Service	Description detaillee	Cas d'usage
Route 53	DNS manage. Traduit les noms de domaine en IP. Supporte les health checks et le failover	Nom de domaine, routage DNS, failover
Lambda	Code serverless. Tu paies par execution (nb requetes + duree).	Traitement d'events, APIs,

	Max 15 min. Triggers: S3, SQS, API Gateway	automatisation
API Gateway	Creer et gerer des API REST/HTTP/WebSocket. Gere throttling, auth, caching	Backend serverless (API GW + Lambda)
Elastic Beanstalk	PaaS : tu upload ton code, AWS gere EC2, ALB, ASG, monitoring. Supporte Node.js, Python, Java, Docker	Deploiement rapide d'app web
ECS / EKS	Executer des conteneurs Docker. ECS = AWS natif, EKS = Kubernetes gere	Microservices conteneurises
Fargate	Executer des conteneurs SANS gerer de serveurs (serverless containers)	Conteneurs sans maintenance EC2
Cognito	Auth utilisateurs (signup, login, MFA). Supporte OAuth, SAML, social login	Auth pour app mobile/web
AWS Secrets Manager	Stocker et auto-rotater les secrets (mots de passe, cles API)	Credentials de BDD
AWS Systems Manager	Gerer les instances (Session Manager, Parameter Store, Patch Manager)	Administration EC2 a distance
Amazon Kinesis	Traitement de donnees en temps reel (streaming)	Logs en temps reel, IoT, analytics
AWS Step Functions	Orchestrer des workflows serverless (Lambda, SQS, etc.)	Processus metier complexes

Architecture Serverless typique

```
Client → API Gateway → Lambda → DynamoDB
```

```
Client → CloudFront → S3 (site statique)
```

Aucun serveur a gerer. Scaling automatique. Pay-per-use.

Route 53 — Routing Policies

Policy	Comportement
Simple	1 domaine → 1 IP (ou plusieurs aleatoirement)
Weighted	Repartir le trafic selon des poids (ex: 70% vers v1, 30% vers v2)
Latency	Envoyer vers la region avec la plus faible latence
Failover	Si le primaire est down → basculer vers le secondaire (health check)
Geolocation	Router selon le pays/continent de l'utilisateur

QCM : Quel service pour creer une API REST serverless qui execute du code Python ?

R : API Gateway + Lambda (Python)

QCM : Comment deployer une app web rapidement sans gerer l'infrastructure ?

R : AWS Elastic Beanstalk (PaaS — tu upload ton code, AWS gere tout)

16. Concepts du mini-projet — Application concrete

Le mini-projet est une **application web Node.js** (Express + EJS + MySQL) deployee sur AWS.

Architecture du projet

```
Utilisateur → Internet → EC2 (Node.js port 80, subnet public, SG: port 80+22)
                                    ↓
                EC2 → Secrets Manager (via IAM Role) → recupere credentials
                EC2 → RDS MySQL (subnet prive, SG: port 3306 depuis SG-EC2 uniquement)
```

Ce qu'il faut retenir pour l'exam

Concept	Comment c'est utilise dans le projet
VPC	Reseau isole avec subnets publics et prives
Subnet public	EC2 avec l'app web (accessible depuis Internet)
Subnet prive	RDS MySQL (pas accessible depuis Internet)
Security Group EC2	Port 80 (HTTP) ouvert a tous, port 22 (SSH) ouvert a mon IP
Security Group RDS	Port 3306 ouvert UNIQUEMENT depuis le SG de l'EC2
IAM Role	Attache a l'EC2, permet de lire les secrets dans Secrets Manager
Secrets Manager	Stocke le host, user, password de la BDD RDS (pas en dur dans le code)
RDS Multi-AZ	Haute disponibilite de la BDD (failover auto)
User Data	Script d'installation de Node.js + npm install au premier boot
systemd service	L'app redemarrera automatiquement en cas de crash

17. MEGA Regles d'or pour les QCM

1 **Haute disponibilite** → Multi-AZ (RDS), ALB + ASG (EC2), S3 (11 neufs)

2 **BDD en securite** → Subnet prive + SG qui n'accepte que le SG du web server

3 **Credentials dans le code** → JAMAIS. Utiliser Secrets Manager ou IAM Role

4 **Site statique** → S3 + CloudFront

5 **Reduire les couts stables** → Reserved Instances / Savings Plans

6 **Reduire les couts non critiques** → Spot Instances

7 **Decoupler** → SQS (1-to-1), SNS (1-to-N)

8 **IaC reproductible** → CloudFormation

9 **Latence mondiale** → CloudFront (CDN)

1 **DNS** → Route 53

1 **Serverless** → Lambda + DynamoDB + S3 + API Gateway

1 **Scaling** → HORIZONTAL (ASG). Presque toujours la bonne reponse

1 **Multi-AZ RDS** = haute dispo | **Read Replica** = performance lecture

1 **SG** = stateful (instance) | **NACL** = stateless (subnet)

1 **EBS** = 1 instance 1 AZ | **EFS** = N instances multi-AZ | **S3** = objets HTTP

1 **Qui a fait quoi ?** → CloudTrail

1 **Metriques et alarmes ?** → CloudWatch

1 **Config conforme ?** → AWS Config

1 **Protection DDoS** → AWS Shield (Standard = gratuit)

2 **Protection appli web** → AWS WAF

2**EC2 sans SSH** → Systems Manager Session Manager

2**Conteneurs sans serveur** → Fargate

2**Trafic entrant AWS** → GRATUIT

2**Responsabilite partagee** → AWS = securite DU cloud / Toi = securite DANS le cloud

2**Si "la reponse la plus economique"** → S3, Lambda, Spot, ou Glacier selon le contexte

Bonne chance pour l'exam !